DailyQuest — Backend Upgrade Summary

Node.js backend restructure + AI features — what's new

This document summarizes the planned upgrade of the DailyQuest app from a front-end-only, localStorage-based app into an app backed by a proper Node.js server, a real database, and AI-powered features. It is meant as a companion to the build prompt given to the development tool (Freebuff), so progress can be tracked milestone by milestone.

1. Why this upgrade

The current version stores everything in the browser's localStorage. That means data is tied to a single browser and device, nothing is backed up, and there is no way to add smarter, server-side features like AI insights. This upgrade moves the app to a client-server architecture so data is safe, synced, and ready for AI features.

2. New backend architecture

- Node.js + Express (or Fastify) REST API server
- Structured folder layout: routes, controllers, services, models, middleware, config
- Database: MongoDB (Mongoose) or PostgreSQL (Prisma) for tasks, users, history, and sessions
- Authentication: JWT-based login/signup, replacing the current fake test-mode login
- Environment-based configuration (.env) for secrets and API keys
- Input validation and centralized error handling middleware
- API versioning (e.g. /api/v1/...) for future-proofing

3. Core API endpoints

Area	Endpoints	Purpose
Auth	POST /auth/signup, /auth/login, /auth/logout	Real user accounts, JWT tokens
Tasks	GET/POST/PUT/DELETE /tasks	Create, edit, delete, and fetch tasks and repeat rules
Progress	POST /tasks/:id/complete, /tasks/:id/progress	Mark complete, update count/timed progress
Focus Sessions	POST /focus/start, /focus/stop, GET /focus/history	Server-logged focus sessions
Reports	GET /reports/daily, /reports/weekly	Generated end-of-day and weekly reports
AI	POST /ai/insights, /ai/suggest-tasks, /ai/chat	AI-powered insights and suggestions
Sync	GET /sync	Pull latest data across devices

4. New AI features

- **Smart daily insights:** instead of raw percentages, the server analyzes history and generates natural-language observations (e.g. patterns in missed tasks, best-performing days).

- **Weekly AI summary:** a short written recap of the week's performance, generated server-side and shown in the Progress screen.
- **Smart task suggestions:** suggests new tasks or adjustments based on completion history (e.g. lowering a target that's rarely met).
- **AI chat / coach (optional):** a simple chat endpoint where the user can ask about their own progress ("How was my week?", "Why do I skip gym?").
- All AI features are powered by a call from the Node.js backend to the Claude API, so the API key stays on the server and is never exposed in the app.

5. Data migration

Existing localStorage data (tasks, XP, streaks, history, focus sessions) will be read on first login after the upgrade and uploaded to the new backend automatically, so no progress is lost. After migration, the app will treat the server as the source of truth, with localStorage used only as an offline cache.

6. What stays the same

- The entire front-end UI, theme, phone frame, and all existing screens
- XP, levels, streaks, badges, repeat rules, task types, and the focus timer logic
- The app continues to work offline using the local cache, and syncs when back online

7. Build order

- 1. Set up the Node.js project structure, database, and environment config
- 2. Build authentication (signup/login/JWT) and replace the fake login
- 3. Build the Tasks, Progress, and Reports APIs; connect the front-end to them
- 4. Migrate existing localStorage data to the backend on first login
- 5. Add the Focus Sessions API and connect the existing timer to it
- 6. Add the AI endpoints (insights, weekly summary, suggestions) using the Claude API
- 7. Add offline caching and background sync

Note: this summary reflects the plan given to the build tool. Actual implementation may vary slightly depending on what the AI coding tool produces — re-check this summary against the real code after each milestone.